

《编译系统原理》总结

第一章 介绍

词法分析中，分词不仅仅是直接分割为子串，还进行了分类。

浏览器是一个广义的编译器。浏览器渲染的过程中是代码生成。

- 抽象语法树：只包含单词
- 具体语法树：包含语法概念

数据库查询也是广义的编译器。

编译器的两个基本组成部分：

- 分析：将源程序分解为基本的组成部分，生成中间表达形式
- 生成：从中间表达形式构建目标程序

格式化打印、格式化编辑器、LATEX、数据压缩等都是编译器。

- 解释器：就是类似于英文中文的翻译，**特点是不产生目标程序，是边分析边执行的。**
 - 举例：翻译，BASIC, Bash, Java, Python
- 编译器：有上下文联系必须编译整个代码

代码优化在前端后端都有。

各阶段干了啥：

- 词法分析：扫描，忽略空格回车等，将字符组成为单词
- 语法分析：生成抽象语法树，将产生的单词组合为语法短语
 - 词法分析不需要递归，语法分析需要
- 语义分析：自底向上
- 中间代码生成：生成llvm语言
- 代码优化：优化代码，分前端后端
- 代码生成：生成可重定位的机器码或者汇编码。确定了变量的内存位置，为变量指定寄存器

符号表：记录了程序的标识符，类型，作用域。**在词法分析时创建**

各阶段错误：

- 词法分析：字符无法构成单词

- 语法分析：单词流违反了语法结构规则
- 语义分析：语法结构正确，但是没有实际意义

所在阶段：涉及到编程语言都是前端，涉及到体系结构都是后端，中端为代码优化

预处理器（不属于编译器的步骤）：

- 宏定义 #define
- #include
- Rational预处理器——是一个举例
- 语言扩展（预处理器的一个功能）

汇编器：由汇编器生成可重定位机器码：

两次扫描汇编：

- 第一次：识别标识符，存入符号表，指定相对内存地址。
- 第二次：产生可重定位机器码。

代码加载就是从相对地址到绝对地址的修改。

连接器：将多个可重定位机器码文件组合成为单一程序。

- 前端：词法分析 语法分析 语义分析 错误处理 中间代码生成 部分中间代码优化
- 后端：部分中间代码优化 代码生成

扫描：多个阶段（词法分析到中间代码生成）合成一次扫描，中间结果全部保存在内存。

第三章 词法分析

设计题

- 正则表达式描述单词

- 描述正规式表示的语言

- ▣ $0^*10^*10^*10^*$: 包含3个1的01串

- ▣ $((\varepsilon | 0)1^*)^*$: 所有01串

- 设计语言的正规式

- ▣ 能被5整除的10进制整数

$[1-9][0-9]^*(0 | 5) | 0 | 5$

- ▣ 包含五个元音，且按顺序排列的所有字母串

- 有限自动机识别单词

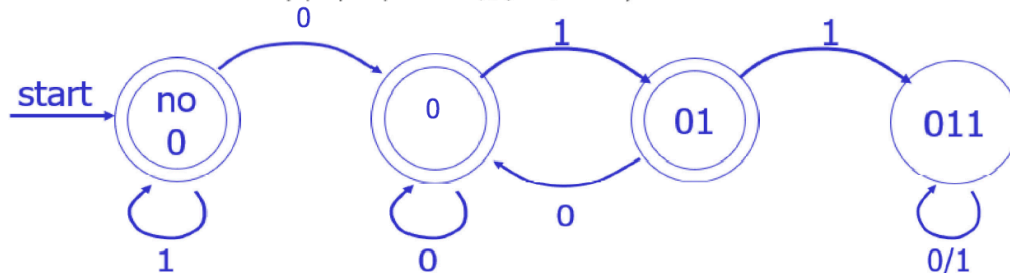
- 一种思路：自动机运转观点

- ▣ 先画初态

- ▣ 状态的变化（画出边及指向状态）——对应符号拼接（边上符号）

- ▣ 符号串变化（当前状态→下一状态）

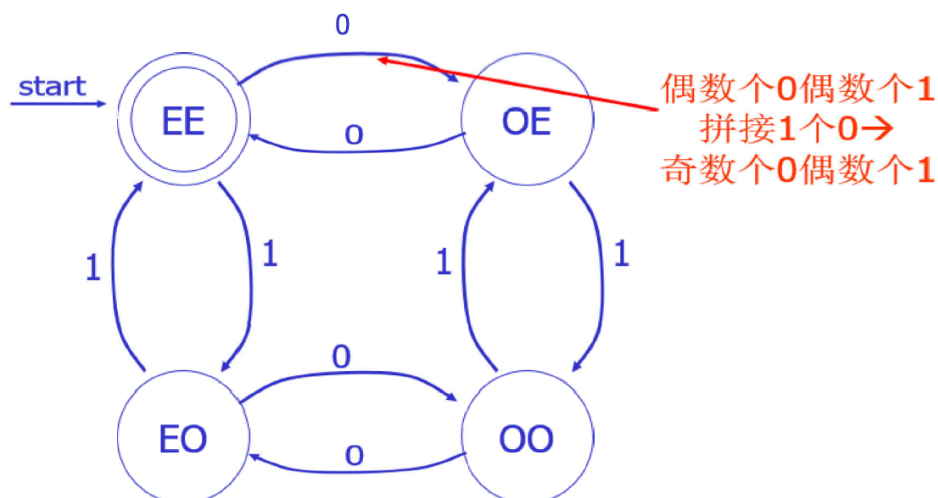
- 不包含子串011的0、1串



- 另一种思路：状态——符号串集合

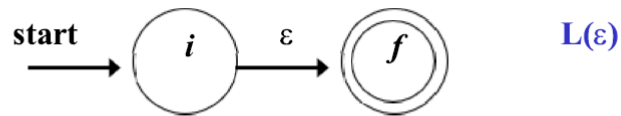
- ▣ 符号串全集划分子集→构造所有状态

- ▣ 状态符号串集合间关系→构造边

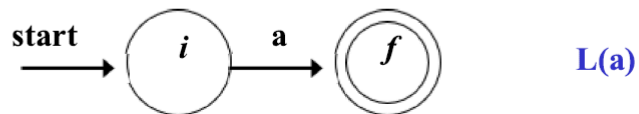


- 正则表达式转NFA——汤普森构造法

- 1. 对于 ϵ ，构造NFA

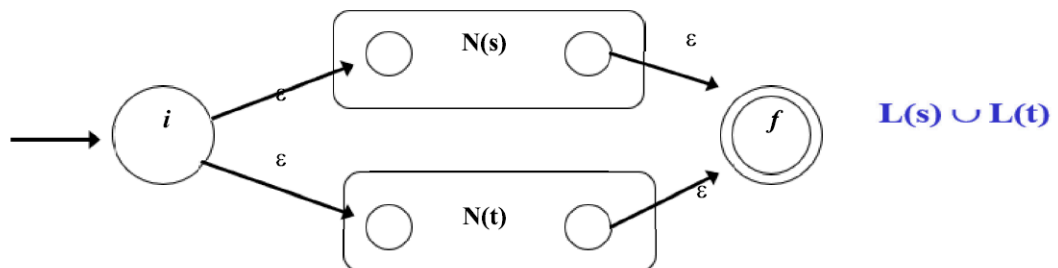


- 2. 对于 $a \in \Sigma$ ，构造NFA



- 3. 假定 $N(s), N(t)$ 是正规式 s, t 对应的NFA

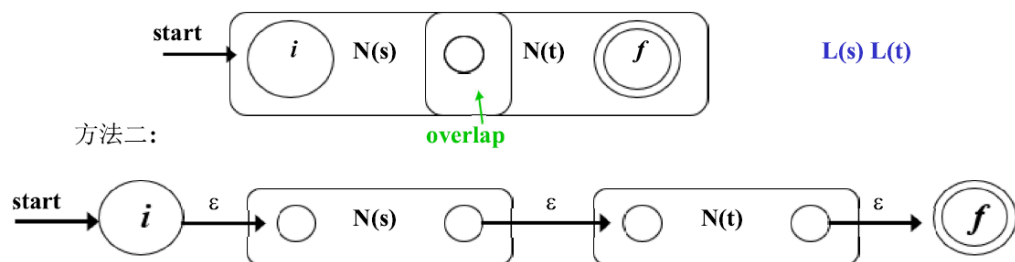
- a. 对正规式 $s|t$ ，构造如下组合NFA $N(s|t)$



两个新状态： i ——新初态 f ——新终态
原初态和终态不再是组合NFA的初态和终态

四条新 ϵ 边： $i \rightarrow N(s)$ 初态 $i \rightarrow N(t)$ 初态
 $N(s)$ 终态 $\rightarrow f$ $N(t)$ 终态 $\rightarrow f$

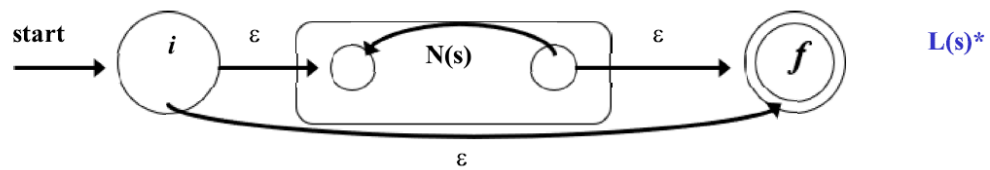
- b. 对正规式 st ，构造如下组合NFA $N(st)$



方法一：新初态—— $N(s)$ 初态，新终态—— $N(t)$ 终态
 $N(s)$ 终态与 $N(t)$ 初态合并

方法二：新初态、终态—— i, f
三条新 ϵ 边： $i \rightarrow N(s)$ 初态 $N(t)$ 终态 $\rightarrow f$
 $N(s)$ 终态 $\rightarrow N(t)$ 初态

- c. 对正规式 s^* ，构造如下组合NFA $N(s^*)$



新初态、终态—— i 、 f

四条新 ϵ 边：
 $i \rightarrow f$
 $i \rightarrow N(s)$ 初态
 $N(s)$ 终态 $\rightarrow f$
 $N(s)$ 终态 $\rightarrow N(s)$ 初态

- d. 对正规式 (s) ， $N((s)) = N(s)$

- NFA转DFA——子集构造法

- NFA $\rightarrow$ DFA的算法

初始， ϵ -closure(s_0)是Dstates (DFA状态集) 中唯一状态，且未标记;

while (Dstates中存在未标记状态T) {

 标记T;

 for (每个输入符号a) {

$U = \epsilon$ -closure($\delta(T, a)$);

 if (U不在Dstates中)

 将U加入Dstates，且设为未标记;

 Dtran[T, a] = U; //DFA状态转换矩阵

 }

}

ϵ -closure(s_0)为初态，包含NFA终态的DFA状态为DFA终态

- DFA最小化——状态划分

- 初始划分，两个组：终态与非终态

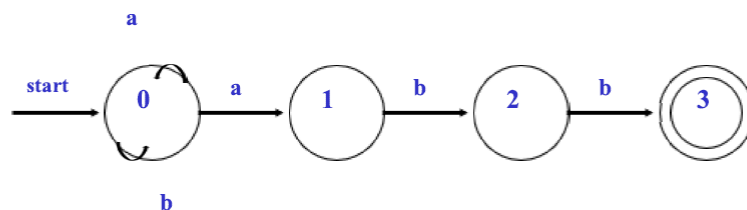
- 分裂方法，对于状态组 $A = \{s_1, s_2, \dots, s_k\}$

□ 对每个符号a，得到其转换状态 $t_1, t_2, \dots, t_k$

□ 若 $t_1, t_2, \dots, t_k$ 属于不同状态组，则需将A对应分裂为若干组

第四章 语法分析

NFA转CFG



1. 状态 $i \rightarrow$ 非终结符 A_i : A_0, A_1, A_2, A_3
 2. $\textcircled{i} \xrightarrow{a} \textcircled{j} \rightarrow A_i \rightarrow aA_j$
 3. $\textcircled{i} \xrightarrow{\varepsilon} \textcircled{j} \rightarrow A_i \rightarrow A_j$
 4. 若 i 为终态 $\rightarrow A_i \rightarrow \varepsilon$: $A_3 \rightarrow \varepsilon$
 5. 若 i 为初态, A_i 为开始符号: A_0
- 正则文法
- $\left\{ \begin{array}{l} : A_0 \rightarrow aA_0, A_0 \rightarrow aA_1 \\ : A_0 \rightarrow bA_0, A_1 \rightarrow bA_2 \\ : A_2 \rightarrow bA_3 \end{array} \right.$

消除左递归

- $\overset{+}{A} \Rightarrow A\alpha$
- 自顶向下分析方法无法处理, 死循环
 - 直接左递归的消除

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

任何 β_i 均不以 A 开头, 改写为:

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$$

$$E \rightarrow E + T \mid T \longrightarrow \begin{cases} E \rightarrow TE' \\ E' \rightarrow + TE' \mid \varepsilon \end{cases}$$

$$T \rightarrow T * F \mid F \longrightarrow \begin{cases} T \rightarrow FT' \\ T' \rightarrow * FT' \mid \varepsilon \end{cases}$$

$$F \rightarrow (E) \mid \text{id} \longrightarrow F \rightarrow (E) \mid \text{id}$$

消除 ε 产生式

- 方法：利用产生式进行代入
- $A \rightarrow \varepsilon, B \rightarrow uAv \Rightarrow B \rightarrow uv \mid uAv$

$$\begin{array}{l} E \rightarrow TE' \\ E' \rightarrow + TE' \mid \varepsilon \\ T \rightarrow FT' \\ T' \rightarrow * FT' \mid \varepsilon \\ F \rightarrow (E) \mid \text{id} \end{array} \left\{ \begin{array}{l} E \rightarrow TE' \mid T \\ E' \rightarrow + TE' \mid + T \\ T \rightarrow FT' \mid F \\ T' \rightarrow * FT' \mid * F \\ F \rightarrow (E) \mid \text{id} \end{array} \right.$$

消除回路

$$S \rightarrow SS \mid (S) \mid \varepsilon$$

$$\text{回路: } S \Rightarrow SS \Rightarrow S$$

$\underbrace{\hspace{1.5cm}}_{S \rightarrow \varepsilon}$

- 如何消除回路？
- 保证每个产生式都加入终结符（开始符号的 ε 产生式除外）
- 上面文法改写为：

$$S \rightarrow S(S) \mid (S) \mid \varepsilon$$

提取左公因子

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

改写为

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

非递归预测分析——LL(1)文法

- 先预处理，消除左递归和提取左公因子
- 求出FIRST集和FOLLOW集

◦ FIRST

- 1. 若X是终结符，则 $\text{FIRST}(X) = \{X\}$
- 2. 若 $X \rightarrow \varepsilon$ ，则将 ε 加入 $\text{FIRST}(X)$
- 3. 若 $X \rightarrow Y_1 Y_2 \dots Y_k$ ，则

$\text{FIRST}(Y_1)$ 加入 $\text{FIRST}(X)$
若 $Y_1^* \Rightarrow \varepsilon$ $\text{FIRST}(Y_2)$ 加入 $\text{FIRST}(X)$
若 $Y_2^* \Rightarrow \varepsilon$ $\text{FIRST}(Y_3)$ 加入 $\text{FIRST}(X)$

...
若 $Y_{k-1}^* \Rightarrow \varepsilon$ $\text{FIRST}(Y_k)$ 加入 $\text{FIRST}(X)$
若 $Y_k^* \Rightarrow \varepsilon$ ε 加入 $\text{FIRST}(X)$

NOTE: 一旦 $Y_i^* \not\Rightarrow \varepsilon$ ，即停止

- 重复1—3，直至所有符号的FIRST集都不再变化

◦ FOLLOW

- 1. $\$$ 加入 $\text{FOLLOW}(S)$ ，S为开始符号， $\$$ 为输入串结束标记
- 2. 若 $A \rightarrow \alpha B \beta$ ，则 $\text{FIRST}(\beta)$ 中符号除 ε 外，均加入 $\text{FOLLOW}(B)$
因为：若有 $\beta^* \Rightarrow a\gamma$ ，显然会有 $S^* \Rightarrow \delta \alpha B a \gamma \eta$
- 3. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta$ 且 $\beta^* \Rightarrow \varepsilon$ ， $\text{FOLLOW}(A)$ 中所有符号加入 $\text{FOLLOW}(B)$

因为：若有 $S^* \Rightarrow \gamma A a \eta$ ，则有 $S^* \Rightarrow \gamma \alpha B a \eta$

- 填表，FIRST直接填，FOLLOW的话就是如果FIRST中出现了空符号的话，就将FOLLOW集写入

Non-terminal	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

- 根据填好的表来进行字符串的入栈分析

STACK	INPUT	OUTPUT
\$E	id + id * id\$	
\$E' T	id + id * id\$	$E \rightarrow TE'$
\$E' T' F	id + id * id\$	$T \rightarrow FT'$
\$E' T' id	id + id * id\$	$F \rightarrow id$
\$E' T'	+ id * id\$	
\$E'	+ id * id\$	$T' \rightarrow \epsilon$
\$E' T+	+ id * id\$	$E' \rightarrow +TE'$
\$E' T	id * id\$	$E' \rightarrow +TE'$
\$E' T' F	id * id\$	$T \rightarrow FT'$
\$E' T' id	id * id\$	$F \rightarrow id$
\$E' T'	* id\$	
\$E' T' F*	* id\$	$T' \rightarrow *FT'$
\$E' T' F	id\$	
\$E' T' id	id\$	$F \rightarrow id$
\$E' T'	\$	
\$E'	\$	$T' \rightarrow \epsilon$
\$	\$	$E' \rightarrow \epsilon$

LL(1)文法的特性:

1. 无二义性，无左递归
2. 若 $A \rightarrow \alpha \mid \beta$ ，则
 - 1) α 、 β 推导出的符号串，不能以同样的终结符a开头
 - 2) α 、 β 至多有一个可推导出 ϵ
 - 3) 若 $\beta \xRightarrow{*} \epsilon$ ， $FIRST(\alpha) \cap FOLLOW(A) = \Phi$

算符优先算法

特点:

- 无 ϵ 产生式
- 产生式右部不含连续的NT

○ 不相交的三种优先级关系—终结符之间

□ $a < b$, a 的优先级低于 b

□ $a \doteq b$, a 的优先级等于 b

□ $a > b$, a 的优先级高于 b

○ 不同于算术关系 $<$ 、 $=$ 、 $>$

□ $a < b$, $a > b$ 可能同时成立

□ 三种关系可能均不成立

○ 确定优先关系

□ 传统意义运算符的优先级: $+ < * , * > +$

算法:

○ 非二义性文法, 语法树反映优先关系

○ 不含 ϵ 产生式文法, 对任一对终结符 a, b

1. $a \doteq b$, 当且仅当 $P \rightarrow \dots ab \dots$ 或 $P \rightarrow \dots aQb \dots$ (同时归约意味着同时计算 $\rightarrow$ 优先级相同)

2. $a < b$, 当且仅当 $P \rightarrow \dots aR \dots$, 且 $R \xRightarrow{+} b \dots$ 或 $R \xRightarrow{+} Qb \dots$
(b 的运算结果 R 是 a 的运算对象 $\rightarrow a$ 优先级低)

3. $a > b$, 当且仅当 $P \rightarrow \dots Rb \dots$, 且 $R \xRightarrow{+} \dots a$ 或 $R \xRightarrow{+} \dots aQ$
(a 的运算结果 R 是 b 的运算对象 $\rightarrow a$ 优先级高)

若算符文法 G 任何终结符对 (a, b) 至多只满足

$a < b, a \doteq b, a > b$

之一, 则称之为算符优先文法

得到算符优先表之后, 我们就可以开始进行字符串的分析了。

STACK	INPUT	Remark
\$	id + id * id\$	\$ < id
\$ id	+ id * id\$	id > +
\$	+ id * id\$	\$ < +
\$ +	id * id\$	+ < id
\$ + id	* id\$	id > *
\$ +	* id\$	+ < *
\$ + *	id\$	* < id
\$ + * id	\$	id > \$
\$ + *	\$	* > \$
\$ +	\$	+ > \$
\$	\$	accept

一系列的弹出操作
——利用某个产生式
进行归约

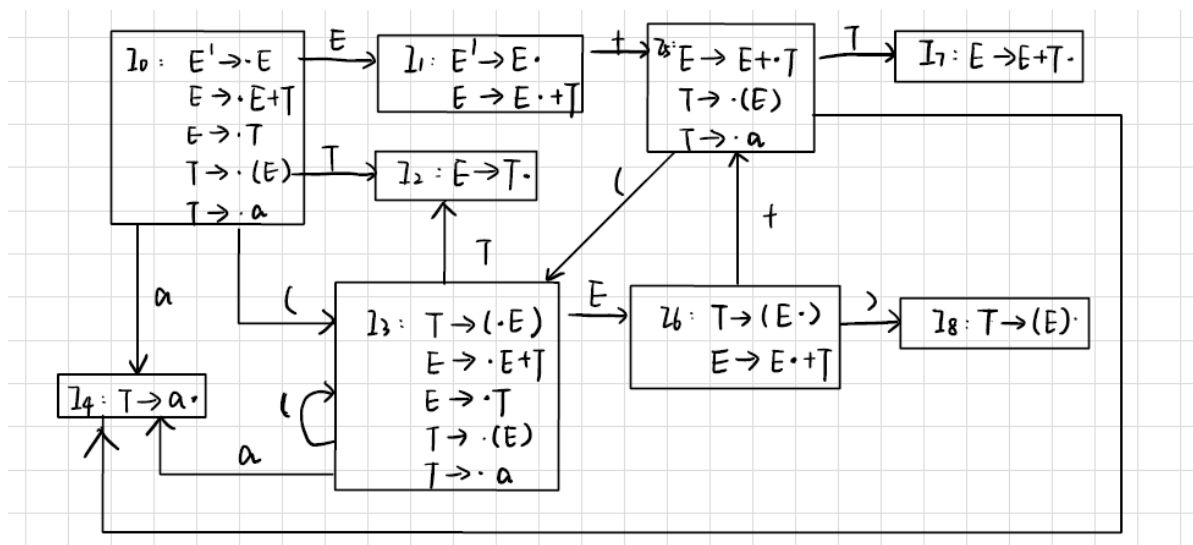
LR(0)构造DFA

先转拓广文法

由已知可得 $G[E']$:

- (0) $E' \rightarrow E$
- (1) $E \rightarrow E+T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow (E)$
- (4) $T \rightarrow a$

就是构造一个项目规范簇（求闭包别忘了）



根据上面的项目簇得到两个表

状态	ACTION 终结符					GOTO 非终	
	a	+	(	)	#	E	T
↓ 0	S ₄		S ₃			1	2
1		S ₅			all		
2	r ₂	r ₂	r ₂	r ₂	r ₂		
3	S ₄		S ₃			6	2
4	r ₄	r ₄	r ₄	r ₄	r ₄		
5	S ₄		S ₃				7
6		S ₅		S ₈			
7	r ₁	r ₁	r ₁	r ₁	r ₁		
8	r ₃	r ₃	r ₃	r ₃	r ₃		

LR(0)构造NFA

2019-2020学年A卷第四大题

还是和前面的构造方法差不多，就是只需要写一条转换了

四、（17分）对下面文法：↵

1. 指出其终结符集合、非终结符集合、开始符号（5分）↵

2. 构造基于 LR(0)项目的识别活前缀的 **NFA**（8分）↵

3. 用此 NFA 对句子 **220** 进行语法分析（4分）。↵

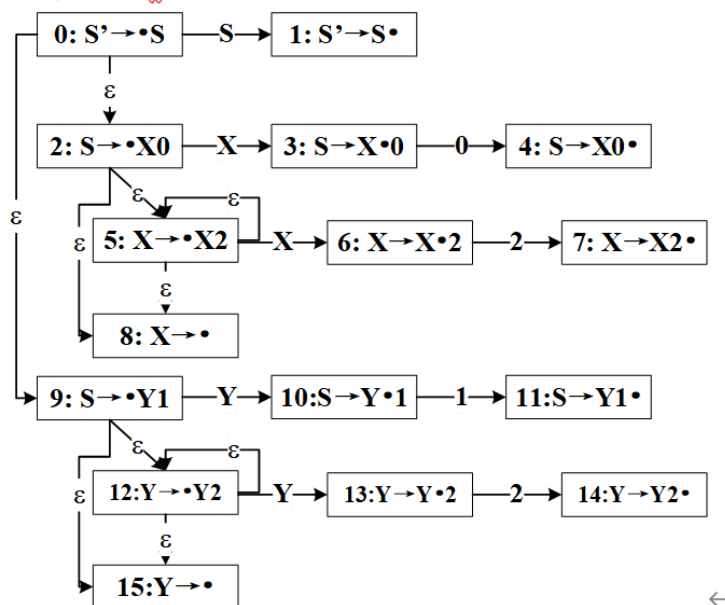
$S \rightarrow X0 \mid Y1$ ↵

$X \rightarrow X2 \mid \varepsilon$ ↵

$Y \rightarrow Y2 \mid \varepsilon$ ↵

解答：

1. 终结符集合 $\{0, 1, 2\}$ ，非终结符集合 $\{S, X, Y\}$ ，开始符号 S
2. 拓广文法 $S' \rightarrow S$



3. 分析 220

初态 $\{0, 2, 5, 8, 9, 12, 15\}$ ，8、15 可归约，15 归约错误路线，8 归约 $X \rightarrow \epsilon$
 $\rightarrow \{3, 6\}$ ，移进 2， $\rightarrow \{7\}$ ，归约 $X \rightarrow X2$
 $\rightarrow \{3, 6\}$ ，移进 2， $\rightarrow \{7\}$ ，归约 $X \rightarrow X2$
 $\rightarrow \{3, 6\}$ ，移进 0， $\rightarrow \{4\}$ ，归约 $S \rightarrow X0$
 $\rightarrow \{1\}$ ，accept

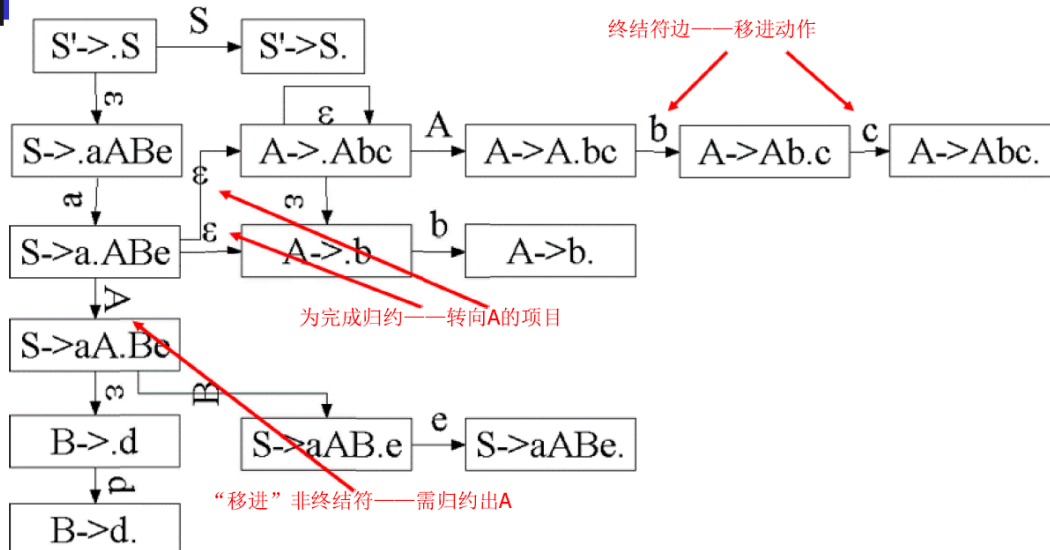
PPT方法：



识别活前缀的NFA

- 拓广文法： $S' \rightarrow S$
- 状态——LR(0)项目， $S' \rightarrow \cdot S$ 为初态
- 项目对应语法栈状态、边对应语法栈动作
 - 状态 $S \rightarrow \cdot aABe$ 转到 状态 $S \rightarrow a \cdot AB e$ (边 'a')
 - 边 'a'——将终结符 'a' 从输入缓冲区移进到语法栈
 - 状态 $S \rightarrow a \cdot AB e$ 转到 状态 $S \rightarrow aA \cdot Be$ (边 'A')
 - 边 'A'——不可能从缓冲区直接移进非终结符 'A'！
 - 必然是通过一系列移进动作（可能还有归约动作），在栈顶形成句柄，最后归约出 'A'！➔ 为了归约出 S，先要归约出 A
 - ➔ 要从状态 $S \rightarrow a \cdot AB e$ 转到 'A' 的产生式的项目！
 $A \rightarrow \cdot Abc$ 、 $A \rightarrow \cdot b$
 - 这些状态转换并未压栈任何符号——'ε' 边！
- 终态？
 - 所有状态！

示例文法的NFA



利用NFA进行LR分析

- NFA运转 $\leftrightarrow$ LR分析：
 - 项目 $\leftrightarrow$ 语法栈（NFA）状态、
 - 语法栈操作 $\leftrightarrow$ NFA中动作（边）
 - 初始时位于NFA初态的 $\epsilon_closure$
 - 读入终结符‘a’
 - NFA状态转换：当前状态集走边‘a’、计算 $\epsilon_closure$
 - LR分析：将‘a’从输入缓冲区移进到语法栈
 - 到达 $S \rightarrow aABe$ 这种状态
 - 已在栈顶形成句柄，归约！
 - 怎么做？归约动作分解：
 - 从栈顶弹出句柄——移进的逆操作——NFA状态转换的逆操作 $\rightarrow$ 沿前几步来时路径回退！
 - 将左部NT压栈——走NT对应边、计算 $\epsilon_closure$ ！

SLR(1)

在LR(0)的基础上多求了一步FOLLOW集，也就是对于整行都是规约的项目，只在其FOLLOW集上写规约

构造文法 $G[S']$ 的 FOLLOW 集：

$$FOLLOW(S') = \{ \# \}$$

$$FOLLOW(E) = \{ +,), \# \}$$

$$FOLLOW(T) = \{ +,), \# \}$$

$$FOLLOW(F) = \{ +, *,), \# \}$$

$$I_1 = \{ + \} \cap FOLLOW(S') = \emptyset$$

$$I_3 = \{ * \} \cap FOLLOW(T) = \emptyset$$

$G[S]$ 是 SLR(1) 文法。

状态	a	+	*	(	)	#	E	T	F
0	S_5			S_4			1	2	3
1		S_6				acc			
2		r_2			r_2	r_2			
3			S_7						
4	S_5			S_4			8	2	3
5		r_6	r_6		r_6	r_6			
6	S_5		S_3	S_4				9	3
7		r_3			r_3	r_3			
8		S_6			S_{10}				
9		r_1			r_1	r_1			
10		r_5	r_5		r_5	r_5			

LR(1)

在 LR(0) 的基础上增加了一个向前搜索符

如何写向前搜索符？

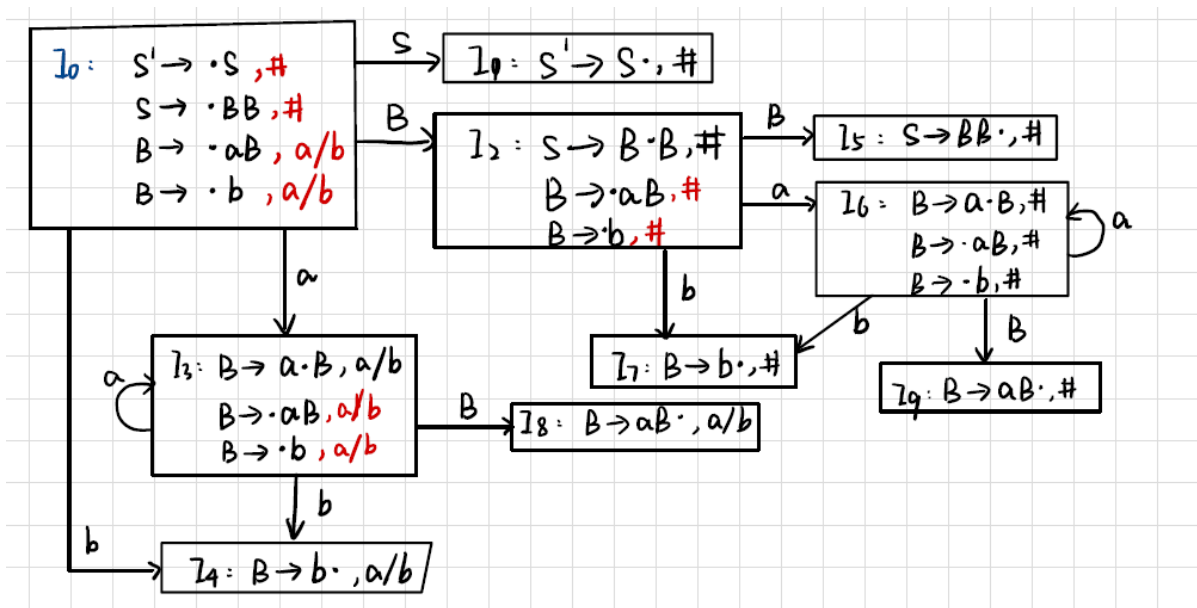
① 文法开始符 Follow 集

② “先看前再看后”

$A \rightarrow \alpha \cdot B \beta$ $\left\{ \begin{array}{l} \beta \text{ 为空, 照抄} \\ \beta \text{ 不为空, } first(\beta) \text{ 加入, 后面.} \end{array} \right.$
 $B \rightarrow \sim$ (β 为非终结符)

就是，先看前面，如果是开始符的话，就直接写 \$，如果是非终结符的话，对应到右边去看，写入所有的点后面是该非终结符的向前搜索符，如果是非终结符后面是空的，就照抄，如果是终结符，就把终结符写进去，如果是非终结符，就写入其 FIRST 集

然后其他部分是一样的



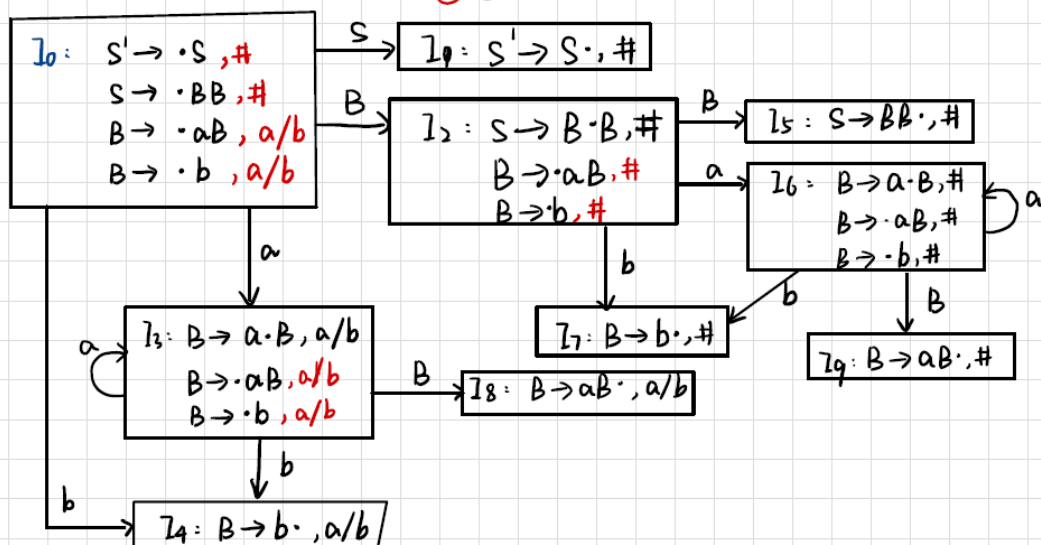
得到两个表:

	ACTION			GOTO	
	a	b	#	S	B
0	S ₃	S ₄		1	2
1			acc		
2	S ₆	S ₇			5
3	S ₃	S ₄			8
4	r ₃	r ₃			
5			r ₁		
6	S ₆	S ₇			9
7			r ₃		
8	r ₂	r ₂			
9			r ₂		

LALR(1)

在LR(1)的基础上，完成了合并同心集的运算，也就是将表达式相同，但是向前搜索符不同的状态进行了合并

(2) (带向前搜索符) 项目集规范族



$I_3, I_6 : I_{36} : B \rightarrow a.B, a/b/\#$
 $B \rightarrow .aB, a/b/\#$
 $B \rightarrow .b, a/b/\#$

$I_4, I_7 : I_{47} : B \rightarrow b., a/b, \#$

$I_8, I_9 : I_{89} : B \rightarrow aB., a/b/\#$

得到两个表:

状态	ACTION			GOTO	
	a	b	#	S	B
0	S ₃₆	S ₄₇		1	2
1			acc		
2	S ₃₆	S ₄₇			5
36	S ₃₆	S ₄₇			89
47	r ₃	r ₃	r ₃		
5			r ₁		
89	r ₂	r ₂	r ₂		

LR分析如何处理字符串

分为三个, 先知道怎么用得到的ACTION表和GOTO表处理字符串

- $\text{action}[s_m, a_i]$: 遇到输入缓冲区中终结符时使用, 四种动作
 1. 移进: 新状态为 s
 2. 归约: 利用产生式 $A \rightarrow \beta$ (再使用 goto 表)
 3. 接受: 输入串合法
 4. 错误: 使用错误恢复机制
- $\text{goto}[s, A]=s'$
 - 归约为 A 时使用, s' 为新状态
- action 表: $s_m \rightarrow s$, goto 表: $s \rightarrow s'$ ——识别活前缀的 **DFA** 的状态转换函数! ——活前缀 (状态) 集合实际上是一个正规集, 可用 **DFA** 识别!
- 初始状态——栈中初始状态
- 格局 (**configuration**)

$$(s_0 X_1 s_1 X_1 \dots X_m s_m, a_i a_{i+1} \dots a_n \$)$$

$$X_1 X_1 \dots X_m a_i a_{i+1} \dots a_n \text{——最右句型}$$

运行方式:

1. $\text{action}[s_m, a_i]=$ 移进 s , 进行移进动作, 格局变化
 $(s_0 X_1 s_1 X_2 \dots X_m s_m a_i s, a_{i+1} \dots a_n \$)$
2. $\text{action}[s_m, a_i]=$ 归约 $A \rightarrow \beta$ ($\beta = X_{m-r+1} \dots X_m$), 进行归约动作, 格局变为
 $(s_0 X_1 s_1 \dots X_{m-r} s_{m-r} A s, a_i a_{i+1} \dots a_n \$)$
 其中, r 为 β 的长度, $s = \text{goto}[s_{m-r}, A]$
3. $\text{action}[s_m, a_i]=$ 接受, 分析结束
4. $\text{action}[s_m, a_i]=$ 错误, 调用错误恢复函数

字符串处理方式:

根据计算出的两个表:

状态	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

来进行字符串的移进与规约

STACK	INPUT	Remark
(1) 0	id * id + id \$	移进
(2) 0 id 5	* id + id \$	归约 $F \rightarrow id$
(3) 0 F 3	* id + id \$	归约 $T \rightarrow F$
(4) 0 T 2	* id + id \$	移进
(5) 0 T 2 * 7	id + id \$	移进
(6) 0 T 2 * 7 id 5	+ id \$	归约 $F \rightarrow id$
(7) 0 T 2 * 7 F 10	+ id \$	归约 $T \rightarrow T * F$
(8) 0 T 2	+ id \$	归约 $E \rightarrow T$
(9) 0 E 1	+ id \$	移进
(10) 0 E 1 + 6	id \$	移进
(11) 0 E 1 + 6 id 5	\$	归约 $F \rightarrow id$
(12) 0 E 1 + 6 F 3	\$	归约 $T \rightarrow F$
(13) 0 E 1 + 6 T 9	\$	归约 $E \rightarrow E + T$
(14) 0 E 1	\$	接受

如何区分LR分析算法

LR(0)文法：不存在冲突项目，也就是移进-规约，规约-规约

如果存在冲突的话，我们就要使用SLR(1)文法了

SLR(1)文法如果存在冲突，就用FOLLOW集来判断，如果为空的话，就说明可以用SLR(1)

如果不为空的话，就说明不能使用

然后使用LR(1)看看

根据向前搜索符去判断，如果搜索符为空的话，就说明是LR(1)文法

如果不为空的话，就说明无法构表

然后尝试LALR文法，合并同心集，就可以了

注意，任何一个SLR文法都是LALR文法

第五章 语法制导翻译

- 综合属性、继承属性
- S - 属性定义
 - 仅综合属性
- L - 属性定义
 - 继承属性依赖父结点，或左兄弟结点
- 语法制导定义/翻译模式的设计

综合属性、继承属性

描述文法符号——③综合属性 ④继承属性

- 综合属性值的计算依赖于分析树中他的子节点的属性值
- 继承属性值的计算依赖于分析树中他的父节点的属性值

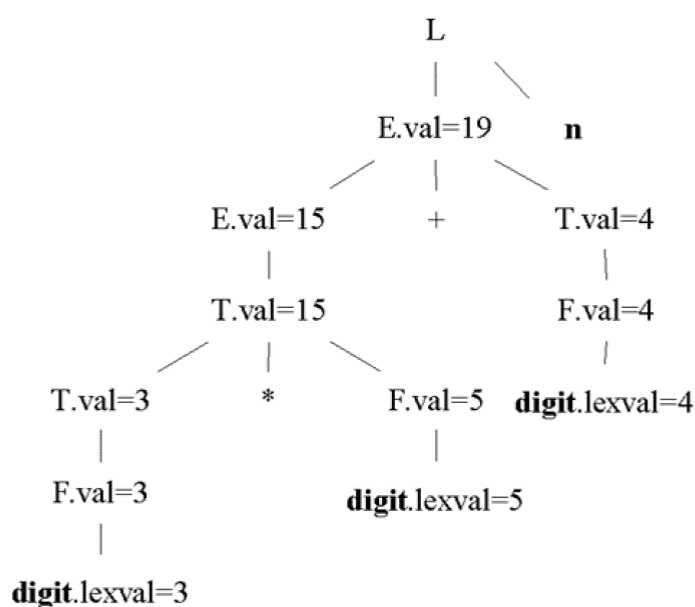
语法制导定义形式：

- 每个产生式 $A \rightarrow \alpha$ 与一组语义规则相关联，每个语义规则具有如下形式：
 - $b = f(c_1, c_2, \dots, c_k)$ ，两种可能情况
 - b 为 A 的综合属性， $c_1, c_2, \dots, c_k$ 为 A 、 α 中语法符号的属性
 - b 为 α 中某个符号的继承属性， $c_1, c_2, \dots, c_k$ 为 A 、 α 中语法符号的属性
 - b 依赖 $c_1, c_2, \dots, c_k$
- **属性文法：**扩充了语法制导定义，无副作用

产生式	语义规则
$L \rightarrow E \mathbf{n}$	$print(E.val)$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

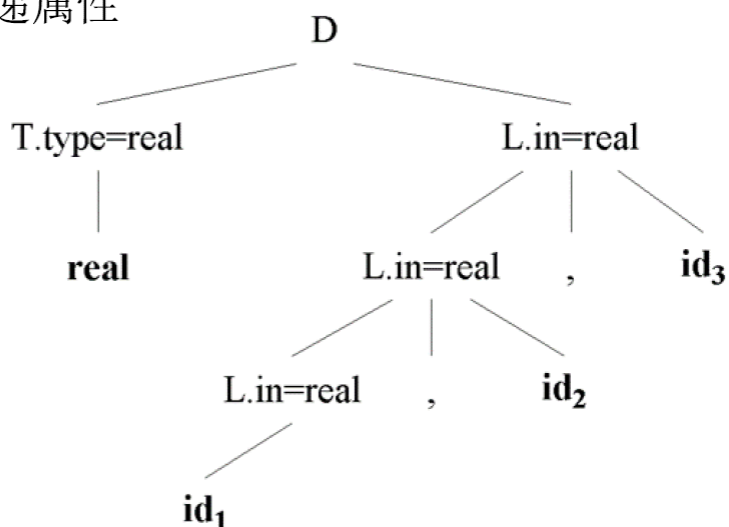
- $\mathbf{digit}.lexval$: 终结符只有综合属性，由词法分析器提供
- 开始符号通常没有继承属性

S属性就是只有综合属性，语法树自底向上进行计算



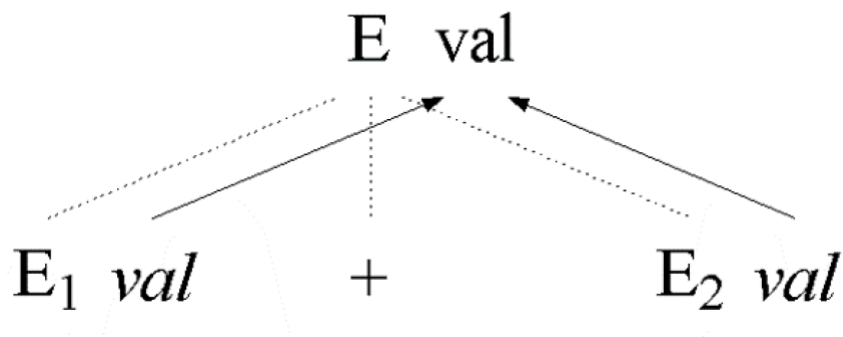
继承属性：就是自上到下的推导

自顶向下传递属性

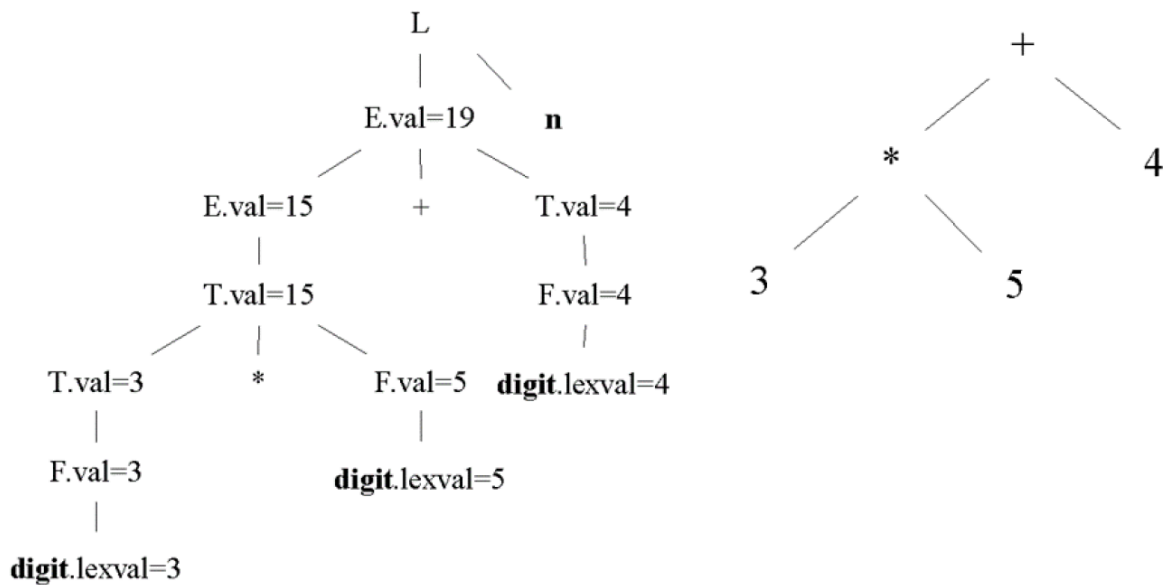


依赖图就是，属性b如果依赖属性a的话，那么属性b应该在a之后进行计算

$$\bigcirc E \rightarrow E_1 + E_2, E.val = E_1.val + E_2.val$$



将语法树进行压缩：



表达式语法树的构造：

- 与表达式翻译为后缀形式类似
- 数据结构：语法树每个节点用一个记录表示

□ 运算符节点记录格式：

```
{  
    运算符  
    指向运算对象节点1的指针  
    执行运算对象节点2的指针  
    ...  
}
```

类似于后缀形式

辅助函数：

mknode(op, left, right): 为运算符op创建语法树中节点，标记为op（运算符），运算对象节点指针left和right

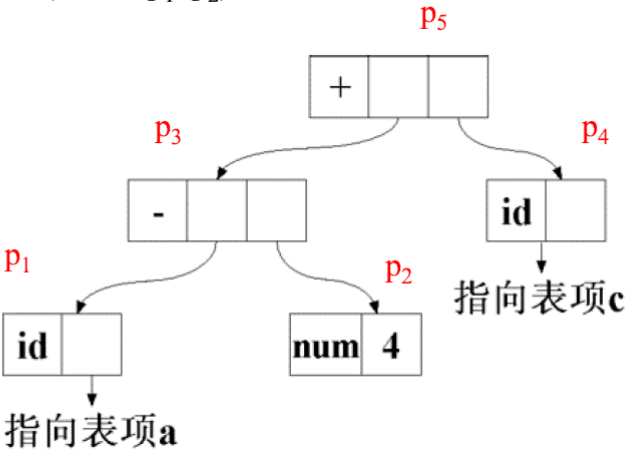
mkleaf(id, entry): 为标识符创建语法树节点，标记为id，另一个域为符号表项指针entry

mkleaf(num, val): 为运算数创建节点，标记为num，另一个域为数值

举个例子：

例5.7 a-4+c的语法树

- (1) $p_1 = \text{mkleaf}(\text{id}, \text{entry_a});$ (4) $p_4 = \text{mkleaf}(\text{id}, \text{entry_c});$
(2) $p_2 = \text{mkleaf}(\text{num}, 4);$ (5) $p_5 = \text{mknode}(' + ', p_3, p_4);$
(3) $p_3 = \text{mknode}(' - ', p_1, p_2);$



对应的语法制导定义：

产生式	语义规则
$E \rightarrow E_1 + T$	$E.nptr = \text{mknode}(" + ", E_1.nptr, T.nptr)$
$E \rightarrow E_1 - T$	$E.nptr = \text{mknode}(" - ", E_1.nptr, T.nptr)$
$E \rightarrow T$	$E.nptr = T.nptr$
$T \rightarrow (E)$	$T.nptr = E.nptr$
$T \rightarrow \text{id}$	$T.nptr = \text{mkleaf}(\text{id}, \text{id.lexval})$
$T \rightarrow \text{num}$	$T.nptr = \text{mkleaf}(\text{num}, \text{num.val})$

S属性：综合属性，自下到上的分析，因此可以和LR(1)分析器相结合

L属性：语法分析同时进行翻译，深度优先的顺序进行属性的计算——适用于LL(1)文法

- 一个语法制导定义，若满足如下条件，则称之为L-属性定义：
 - 对产生式 $A \rightarrow X_1 \dots X_n$ ， $X_j (1 \leq j \leq n)$ 的每个继承属性应仅仅依赖于
 1. X_j 左边符号 $X_1, X_2, \dots, X_{j-1}$ 的属性（继承属性和综合属性）
 2. A 的继承属性
- 所有S-属性定义都是L-属性定义

所有的S都是L!!!

设计翻译模式

- 以语法制导定义为基础
- 注意L-属性定义所带来的限制，确保不会违反属性计算的依赖关系
- 最简单情况：只有综合属性
- 既有综合属性，又有继承属性
 1. 一个符号的继承属性必须在符号之前（左边）的语义动作中进行计算
 2. 动作不能引用它右边符号的综合属性
 3. 产生式左部NT的综合属性，必须在其依赖的所有属性计算完毕后，才能计算。一般置于右部的末尾

例题：

下面文法描述了正则表达式，设计语法制导定义实现构造正则表达式对应的表达式树。假设已有辅助函数mkleaf(char) (及mkleaf_epsilon())和mknode(op, child1, child2)分别为基本正则表达式和正则表达式运算创建叶节点和内部节点

$R \rightarrow \text{char} \mid ' \epsilon ' \mid R ' \mid R \mid R . R \mid R * \mid (R)$

答案:

```
R → char      { R.p = mkleaf(char.lexme); }
R → ' ε '     { R.p = mkleaf_epsilon(); }
R → R.1 ' | ' R2 { R.p = mknode(UNION, R.1.p, R2.p); }
R → R.1 . R2 { R.p = mknode(CON, R.1.p, R2.p); }
R → R.1 * { R.p = mknode(CLOSURE, R.1.p, NULL); }
R → ( R.1 ) { R.p = R.1.p; }
```

←

得 分

←

五、 (10 分) 对下面描述后缀表达式的文法设计语法制导定义，实现后缀表达式到中缀表达式的转换，要求无多余括号。←

$E \rightarrow E E + \mid E E * \mid \text{num}$ ←

←

←

←

答: $E \rightarrow E_1 E_2 + \{ \text{if } (E_2.op == "+") E.t = E_1.t \parallel "+" \parallel E_2.t \parallel "(" \};$ ←
 $\text{else } E.t = E_1.t \parallel "+" \parallel E_2.t;$ ←
 $E.op = "+"; \}$ ←

$E \rightarrow E_1 E_2 * \{ \text{if } (E_1.op == "+") E.t = "(" \parallel E_1.t \parallel ")" * "(" \parallel E_2.t \parallel ")" ;$ ←
 $\text{else } E.t = E_1.t \parallel "*" \parallel E_2.t \parallel "(" ;$ ←
 $E.op = "*"; \}$ ←

$E \rightarrow \text{num} \{ E.t = \text{itoa}(\text{num.val}); E.op = "id"; \}$ ←

8. 下面语法制导定义是____ E ____，其计算容易与____ E ____相结合。←

$A \rightarrow QR$

$R.i = f_4(A.i); Q.i = f_5(R.s); A.s = f_6(Q.s);$ ←

A. S-属性定义

B. L-属性定义←

C. 预测分析法

D. SLR 分析法←

E. 以上皆错←

7. 下面语法制导定义是____ E ____，其中 i 是____ B ____、 s 是____ A ____。←

$S \rightarrow AB \{ B.i = f_1(S.i); A.i = f_2(B.s); S.s = f_3(A.s) \}$ ←

A. 综合属性

B. 继承属性←

C. S-属性定义

D. L-属性定义←

E. 以上皆错←

得分
←

五、 (11 分) 写出描述后缀算术表达式(支持基本表达式 NUM 和 +、* 两种运算) 的 CFG, 为其设计语法制导定义, 转换为中缀形式的表达式, 要求无多余括号、不能改变原有计算次序(数学上等价也不可)。

答: 如括号冗余减 3 分, 没有正确加括号减 6 分, 其他酌情减分

```

E → E E +    { if (E2.op == PLUS)
                  E.t = E1.t + "(" + E2.t + ")";
                  else E.t = E1.t + "+" + E2.t;
                  E.op = PLUS;
                }
| E1 E2 *    { if (E1.op == PLUS)
                  E.t = "(" + E1.t + ")" * "(" + E2.t + ")";
                  else E.t = E1.t + "*" + E2.t;
                  E.op = MUL;
                }
| NUM        { E.t = itoa(NUM.val); E.op = NOP; }

```

得分
←

五、 语法制导翻译 (10 分)

写出描述类型表达式 (支持基本类型 char、int、real、bool 以及类型构造符 array、×、record、pointer 和 →) 的 CFG, 为其设计语法制导定义, 构造对应的抽象语法树 (叶节点为基本类型、非叶节点为类型构造符, 假设已有创建叶节点的辅助函数 mkleaf 和创建非叶节点的辅助函数 mknode, 在语法制导定义中直接调用即可)。

答:

```

T → array (' I, T1 ')    { T.nptr=mknode("array", I.nptr, T1.nptr); }
| T1 '×' T2              { T.nptr=mknode("×", T1.nptr, T2.nptr); }
| pointer (' T1 ')        { T.nptr=mknode("pointer", T1.nptr); }
| record (' T1 ')         { T.nptr=mknode("record", T1.nptr); }
| T1 '→' T2              { T.nptr=mknode("→", T1.nptr, T2.nptr); }
| boolean                 { T.nptr=mkleaf("boolean"); }
| char                    { T.nptr=mkleaf("char"); }
| integer                 { T.nptr=mkleaf("integer"); }
| real                    { T.nptr=mkleaf("real"); }

I → num1 .. num2         { I.nptr=mkleaf("range", num1.val, num2.val); }

```

得分

←

五、 (16分) ←

(1) 设计上下文无关文法描述类型表达式 (注意, 不是描述类型), 基本类型为 **integer**、**char**、**real**, 类型构造符为 **array**、**pointer**、**→** (函数) 和 **X** (笛卡儿积)。(6分) ←

(2) 设计语法制导定义实现类型表达式转换为紧凑二进制编码表示, **integer**、**char**、**real** 编码分别为 01、10、11, **array**、**pointer**、**→** (函数) 编码分别为 01、10、11。对类型表达式 **array(num, pointer(char)→integer)** (数组的索引集合用一个常数表示), 画出其语法树, 利用你设计的语法制导定义对语法树中节点计算属性值, 给出最终的翻译结果 (10分) ←

←

答: ←

(1) 分数划分: 4 个类型构造符各 1 分, base_type 2 分 ←

←

```
type_exp → base_type | array (num , type_exp) ←  
          | pointer (type_exp) | type_exp 'X' type_exp ←  
          | type_exp '→' type_exp ←  
base_type → integer | char | real ←
```

←

(2) 分数划分: 语法制导定义 6 分 ←

←

```
type_exp → base_type { type_exp.code = base_type.code; } ←  
          | array (num , type_exp1) { type_exp.code = "01" ||  
type_exp1.code; } ←  
          | pointer (type_exp1 ) { type_exp.code = "10" ||  
type_exp1.code; } ←  
          | type_exp1 'X' type_exp2 { type_exp.code =  
type_exp1.code; } ←  
          | type_exp1 '→' type_exp2 { type_exp.code = "11" ||  
type_exp2.code; } ←  
base_type → integer { base_type.code = "01"; } ←  
          | char { base_type.code = "10"; } ←  
          | real { base_type.code = "11"; } ←
```


得分
↵

四、（16 分）对下面前缀表达式文法↵

$E \rightarrow + E E \mid - E E \mid * E E \mid / E E \mid \text{num}$ ↵

↵

↵

(1) 列出终结符、非终结符和开始符号（3 分）↵

(2) 文法中没有描述带括号的表达式，是否不完整？为什么？

(3 分) ↵

(3) 设计语法制导定义将其转换为中缀表达式（注意保持正确的运算顺序），画出*9+52 的语法树，计算属性值将其扩展为注释语法树（10 分）↵

答：↵

(1) 终结符：+、-、*、/、num↵

非终结符：E 开始符号：E↵

↵

*一个 1 分↵

↵

(2) 不是，因为前缀表达式无需括号即可表达正确计算顺序↵

↵

*结论 1 分，解释 2 分↵

↵

(3) ↵

$E \rightarrow + E_1 E_2 \quad \{ E.\text{prefix} = "(" \parallel E_1.\text{prefix} \parallel "+" \parallel E_2.\text{prefix} \parallel ")"; \}$ ↵

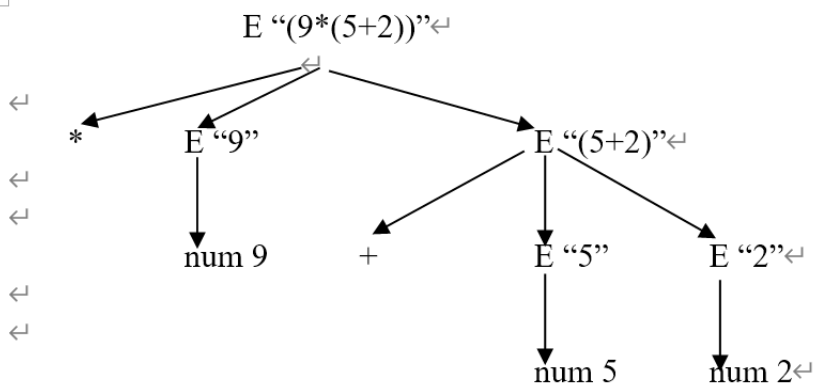
$E \rightarrow - E_1 E_2 \quad \{ E.\text{prefix} = "(" \parallel E_1.\text{prefix} \parallel "-" \parallel E_2.\text{prefix} \parallel ")"; \}$ ↵

$E \rightarrow * E_1 E_2 \quad \{ E.\text{prefix} = "(" \parallel E_1.\text{prefix} \parallel "*" \parallel E_2.\text{prefix} \parallel ")"; \}$ ↵

$E \rightarrow / E_1 E_2 \quad \{ E.\text{prefix} = "(" \parallel E_1.\text{prefix} \parallel "/" \parallel E_2.\text{prefix} \parallel ")"; \}$ ↵

$E \rightarrow \text{num} \quad \{ E.\text{prefix} = \text{itos}(\text{num.val}); \}$ ↵

↵



*语法制导定义 5 分，注释语法树 5 分↵

若设计的是翻译模式（打印数、运算符和括号，而不是计算中缀字符串保存在属性中），完全正确（翻译模式、带语义动作的语法树以及最后的中缀表达式都正确给出了）给 7 分。↵

部分正确的情况在上述基础上酌情扣分。↵

↵

4. 根据语法规则定义, 实现三运算表达式求值的翻译. 基本运算表达式构造 0 和 1. 运算符优先级与位或、取反. 输出 1 的翻译语法树, 并给出最终翻译结果.

运算符: $\&$, $\sim$. 括号这些符号.

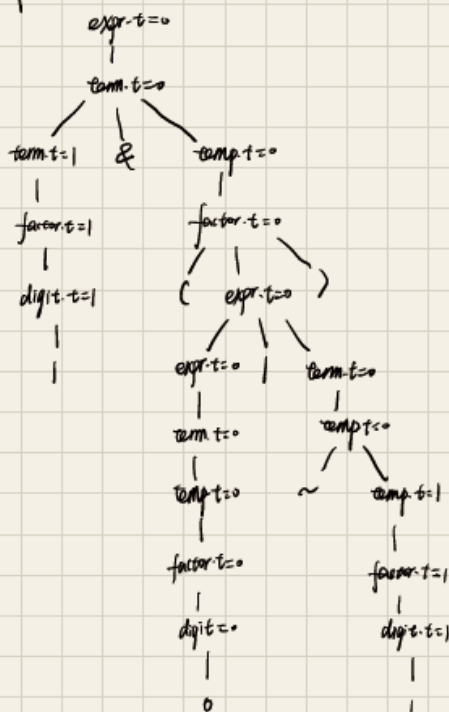
产生式: $\begin{aligned} \text{expr} &\rightarrow \text{expr} \& \text{term} \mid \text{term} \\ \text{term} &\rightarrow \text{term} \& \sim \text{term} \mid \text{temp} \\ \text{temp} &\rightarrow \sim \text{temp} \mid \text{factor} \\ \text{factor} &\rightarrow (\text{expr}) \mid \text{digit} \\ \text{digit} &\rightarrow 0 \mid 1 \end{aligned}$

语法规则: $\begin{aligned} \text{expr.t} &= \text{expr.t} \& \text{term.t} \\ \text{expr.t} &= \text{term.t} \\ \text{term.t} &= \text{term.t} \& \sim \text{temp.t} \\ \text{term.t} &= \text{temp.t} \\ \text{temp.t} &= \sim \text{temp.t} \\ \text{temp.t} &= \text{factor.t} \\ \text{factor.t} &= \text{expr.t} \\ \text{factor.t} &= \text{digit.t} \\ \text{digit.t} &= '0' \\ \text{digit.t} &= '1' \end{aligned}$

输出 $(0 \mid \sim 1)$ 的翻译语法树

$\begin{aligned} \text{expr} &\rightarrow \text{term} \\ &\rightarrow \text{term} \& \text{temp} \\ &\rightarrow \text{temp} \& \text{temp} \\ &\rightarrow \text{factor} \& \text{temp} \\ &\rightarrow \text{digit} \& \text{temp} \\ &\rightarrow 1 \& \text{temp} \\ &\rightarrow 1 \& \text{factor} \\ &\rightarrow 1 \& (\text{expr}) \\ &\rightarrow 1 \& (\text{expr} \mid \text{term}) \\ &\rightarrow 1 \& (\text{term} \mid \text{temp}) \\ &\rightarrow 1 \& (\text{temp} \mid \text{temp}) \\ &\rightarrow 1 \& (\text{factor} \mid \text{temp}) \\ &\rightarrow 1 \& (\text{digit} \mid \text{temp}) \\ &\rightarrow 1 \& (0 \mid \text{temp}) \\ &\rightarrow 1 \& (0 \mid \sim \text{temp}) \\ &\rightarrow 1 \& (0 \mid \sim \text{factor}) \\ &\rightarrow 1 \& (0 \mid \sim \text{digit}) \\ &\rightarrow 1 \& (0 \mid \sim 1) \end{aligned}$

构建翻译语法树:



第六章 类型检查

类型表达式

就是用来表达语言结构的类型的表达式

1. 基本类型: *boolean, char, integer, real, type_error, void*
2. 类型名

3. 类型构造符

- a. 数组：T是类型表达式，I为索引集合（整数范围），则 $\text{array}(I, T)$ 是一个类型表达式，表示元素为类型T的数组类型

$\text{int } A[10];$ —— $\text{array}(\{0, \dots, 9\}, \text{integer})$

- b. 笛卡儿积： T_1 、 T_2 为类型表达式，则 $T_1 \times T_2$ 为类型表达式

-
- c. 记录：与笛卡儿积的不同之处仅在于记录的域有名字。<域名，域类型>元组

```
typedef struct {  
    int address;  
    char lexeme[15];  
} row;
```

$\text{row table}[101];$

类型表达式为：

$\text{record}((\text{address} \times \text{integer}) \times$
 $(\text{lexeme} \times \text{array}(\{0, \dots, 15\}, \text{char})))$

- d) 指针：T为类型表达式，则 $\text{pointer}(T)$ 为类型表达式，表示“指向类型为T的对象的指针”类型

$\text{row } *p;$ —— $\text{pointer}(\text{row})$

- e) 函数：数学上，一个集合“定义域”到另一个集合“值域”的映射。程序语言，定义域类型D到值域类型R的映射： $D \rightarrow R$ 。

➤ %运算符

$(\text{int} \times \text{int}) \rightarrow \text{int}$

➤ $\text{int } *f(\text{char } a, \text{char } b);$

$(\text{char} \times \text{char}) \rightarrow \text{pointer}(\text{integer})$

➤ $(\text{integer} \rightarrow \text{integer}) \rightarrow (\text{integer} \rightarrow \text{integer})$

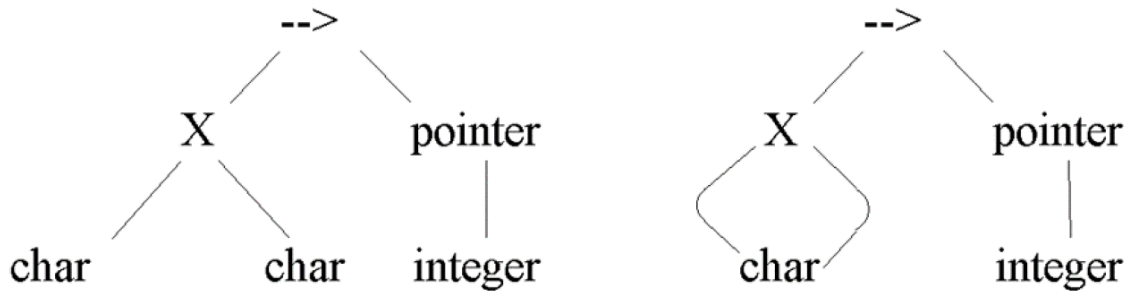
函数类型，参数和返回值都是接受一个整型、返回一个整型的函数

4. 可使用类型表达式变量



图表示类型表达式

○ $(\text{char} \times \text{char}) \rightarrow \text{pointer}(\text{integer})$



类型表达式等价

两种等价：结构等价和名字等价

结构等价：

结构等价

- 类型表达式的图表示（叶结点：基本类型、类型名，内部节点：类型构造符）是否相同
- 递归定义类型会导致回路
- 结构等价（**structural equivalence**）
 1. 相同的基本类型
 2. 对子表达式施加的类型构造符相同
 - 两个类型表达式结构等价——dag中对应相同结点
- 有时要放松条件——数组参数忽略下标范围
 - 等价性检查算法稍做修改

等价性检查算法

```
bool sequiv(s, t)
{
    if (s和t为相同基本类型)
        return true;
    else if (s == array(s1, s2) and t == array(t1, t2))
        return sequiv(s1, t1) and sequiv(s2, t2);
    else if (s == s1 × s2 and t = t1 × t2)
        return sequiv(s1, t1) and sequiv(s2, t2);
    else if (s == pointer(s1) and t == pointer(t1))
        return sequiv(s1, t1);
    else if (s == s1 → s2 and t == t1 → t2)
        return sequiv(s1, t1) and sequiv(s2, t2);
    else return false;
}
```

名字等价:

```
type link = ^cell;
var next : link;
    last : link;
    p : ^cell;
    q, r : ^cell;
```

- 5个变量类型是否都相同? ——依赖具体编译器实现采用哪种类型等价判定
 - 类型表达式允许类型名, 但不允许回路
 - 名字等价: 类型表达式完全相同
 - 结构等价: 类型名被替换后完全相同
-
- 名字等价: 类型表达式完全相同
 - 结构等价: 类型名被替换之后完全相同

变量	类型表达式
next	link
last	link
p	pointer(cell)
q	pointer(cell)
r	pointer(cell)

- 结构等价：5个变量类型都相同
- 名字等价：next, last类型相同，p, q, r类型相同，p, next类型不同

- C——对函数结构等价。**对struct名字等价**。对record也是名字等价
- Pascal——默认名字等价。
- 也就是说，对于struct来说，C和pascal都是名字等价的

关于下面类型表达式，正确说法是_____。
(pointer(char)→int)→pointer(char)

- ☐ A C语言对这种类型的等价判定采用名字等价方式
- ☐ B Pascal语言对这种类型的等价判定采用结构等价方式
- ☒ C 在C语言中，这种类型会引发类型错误
- ☐ D 以上皆错

第七章 运行时环境

考试要求：

运行时环境

- 内存分配策略：静态、栈式、堆式
 - 具体内存组织方式
 - 如何处理函数调用
- 函数参数传递方式与内存分配间的关系

内存分配策略

1. 静态分配（static）

- 在编译时（运行之前）确定所有数据的内存分布

2. 栈分配（stack）

- 利用栈管理运行时存储

3. 堆分配（heap）

- 允许用户在运行时动态地在堆之上分配、释放内存

静态分配：不允许递归，不允许动态的数据分配

7.3.1 静态分配策略

○ 局限性

1. 数据对象的大小和对其内存位置的限制，必须在编译时已知
2. 不允许递归←
对过程的多次活动，局部名字都绑定到相同的存储位置——局部名字的值在多次调用间会得到保持（retain）
3. 不允许动态数据分配

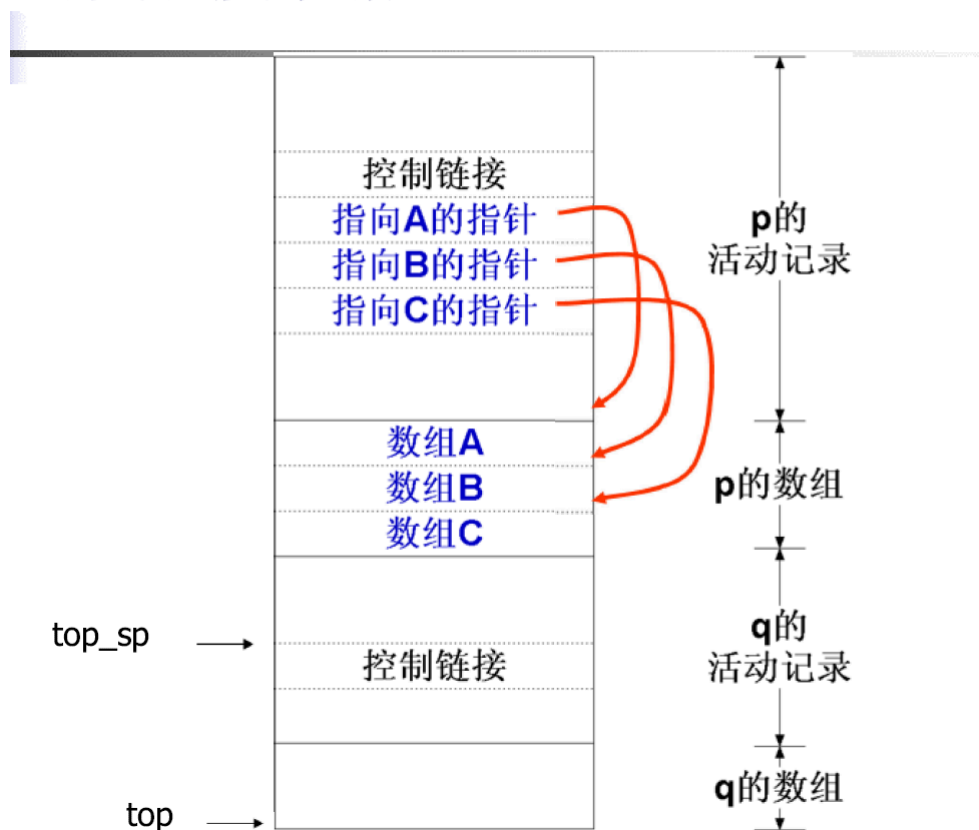
栈分配：

7.3.2 栈分配策略

- 过程调用——push 活动记录
- 过程返回——pop 活动记录
- 每次过程活动，局部名字都绑定到新的位置
- 过程结束，局部名字空间被释放——值丢失

注意变量的作用域问题

可变长度数据



参数传递

传值方式

○ 7.5.1 传值方式, call-by-value

- 形参与局部名字同样处理 → 存储位置在被调函数的活动记录中
- 调用函数计算实参值, 将其右值放置到形参的存储位置
- 对形参的操作不会影响实参的值

传递的例子:

```
void swap(int x, int y)
{
    int temp;

    temp = x; x = y; y = temp; }

```

```
void main()

```

```
{
    int a = 1, b = 2;

    swap(a, b);

    printf( "a is now %d, b is
           now %d\n" , a, b); }

```

结果

a is now 1, b is now 2

调用者 存储空间	swap 存储空间
...	...
a: 1	x: 2
b: 2	y: 1
...	temp: 1
	...

对形参的操作不会影响实参的值, 也就是说, 调用者存储空间中的数据发生变化, 是不会影响主存储空间中的数据的

也可以通过传递指针来改变实参的值!!!

通过传递指针改变实参值

```
void swap(int *x, int *y)
```

```
{
```

```
    int temp;
```

```
    temp = *x; *x = *y; *y = temp;
```

```
}
```

```
void main()
```

```
{
```

```
    int a = 1, b = 2;
```

```
    swap(&a, &b);
```

```
    printf( "a is now %d, b is now %d\n" , a, b);
```

```
}
```

显式传递地址

结果

a is now 2, b is now 1

传地址方式

还可以隐式传参：

○ call-by-reference, call-by-address,

call-by-location, 隐含传递地址

- 实参是名字或表达式，且具有左值，则其左值将被传递给形参
- 实参无左值，如a+b或2，则为其分配新的空间，将此空间的地址传递给形参
- 形参与实参——同一个数据对象
- 引用形参——用传递来的指针进行间接引用

```
program reference(input, output)
```

```
var a, b : integer;
```

```
procedure swap(var x, y : integer);
```

```
var temp : integer;
```

```
begin
```

```
temp := x; x := y; y := temp;
```

```
end;
```

```
begin
```

```
a := 1; b := 2;
```

```
swap(a, b);
```

```
writeln( 'a = ', a); writeln( 'b = ', b);
```

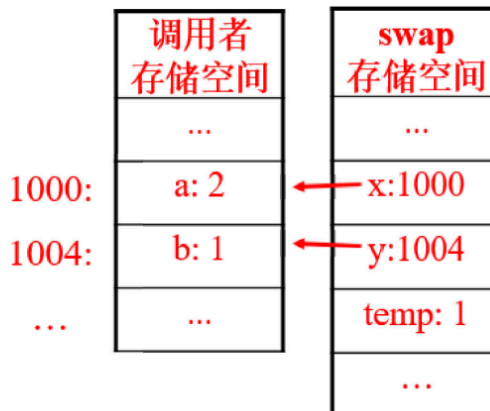
```
end.
```

结果

a = 2

b = 1

传地址方式



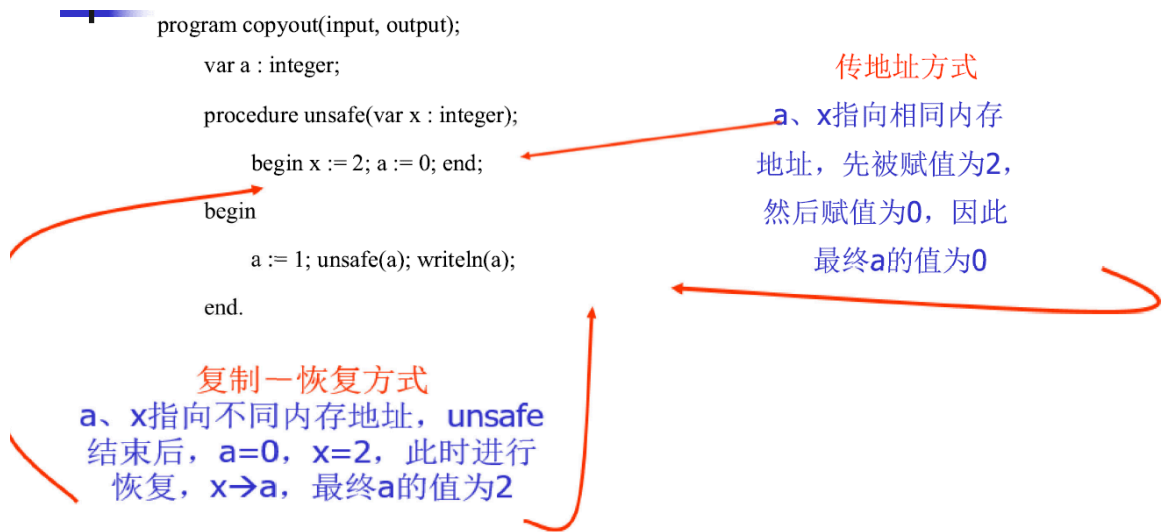
○ 若调用swap(i, a[i])

1. 将i和a[i]的左值复制到swap的活动记录中，假定保存在arg1和arg2两个位置——分别对应形参x和y
2. temp:=x——将arg1指向位置的值（i的初值，假定为I₀）赋予temp
3. x:=y——将arg2指向位置的值赋予arg1指向的位置——i := a[I₀]
4. y:=temp——将temp的值赋予arg2指向的位置——a[I₀]:=I₀

复制-恢复方式

○ copy-restore，传值和传地址的混合， copy-in， value-result

1. 进入被调用函数之前，计算实参值，将其右值传送到被调用函数（传值）。另外，若实参有左值的话，还要求出左值
2. 当调用返回时，将形参当前的右值复制回实参左值指向位置。



传名方式

○ Algol，拷贝规则，copy-rule

1. 过程像宏一样处理

- 被调用——用过程体程序文本替换过程名
- 实参的程序文本替换形参
- 宏扩展，marco-expansion
- 内联扩展，in-line expansion

2. 被调用过程内的局部名字保证与调用过程中的名字不混淆——可认为这些局部名字在宏扩展前改名为独一无二的名字

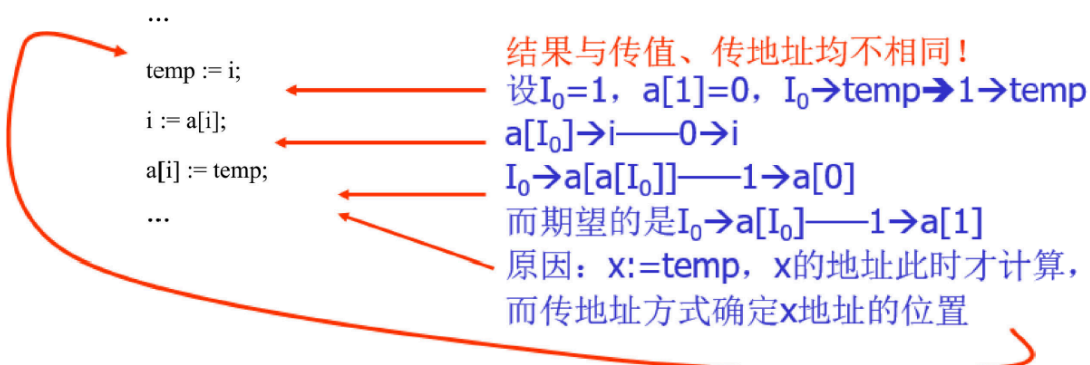
3. 实参可能被加以括号，以保证完整性

```

...
swap(i, a[i]);
...

```

○ 在传名方式下，实际上相当于如下代码



第八章 中间代码生成

考试要求:

- 表达式翻译
 - 临时名字重用
- 布尔表达式和控制流语句的翻译
 - 真假值出口

三地址码

○ Three-Address Code

- 一般形式: $x := y \text{ op } z$
- 语法树、dag的线性化表示

$t_1 := -c$	$t_1 := -c$
$t_2 := b * t_1$	$t_2 := b * t_1$
$t_3 := -c$	$t_5 := t_2 + t_2$
$t_4 := b * t_3$	$a := t_5$
$t_5 := t_2 + t_4$	
$a := t_5$	

语句类型:

1. 二元运算: $x := y \text{ op } z$
2. 一元运算: $x := \text{op } y$
3. 复制语句: $x := y$
4. 无条件转移: $\text{goto } L$
5. 条件转移: $\text{if } x \text{ relop } y \text{ goto } L$
6. 函数调用: $\text{param } x_1$
 $\text{param } x_2$
 $\dots$
 $\text{param } x_n$
 $\text{call } p, n$
7. 数组: $x := y[i]$, $x[i] := y$
8. 指针: $x := \&y$, $x := *y$

homework:

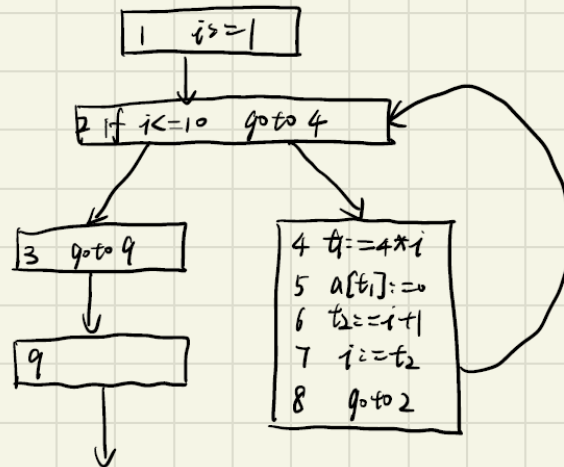
1. 下面C程序翻译为三地址码，画出流图

```
main()
{
    int i;
    int a[10];
    i = 1;
    while (i <= 10) {
        a[i] = 0; i = i + 1;
    }
}
```

翻译为三地址码

```
1 i:=1
2 if i<=10 goto 4
3 goto 9
4 t1:=4*i
5 a[t1]:=0
6 t2:=i+1
7 i:=t2
8 goto 2
9
```

流图



表达式翻译

临时变量重用

- newtemp 每次产生不同名字 → 临时变量数 = 表达式中运算符数，空间占用巨大！
- 临时变量保存的中间计算结果，后续不会继续使用，浪费空间！
- 能否重用临时变量保存不同中间结果？ ← 分析变量生存周期：语法制导翻译方法、基于中间代码进行优化
- $E \rightarrow E_1 + E_2 \rightarrow$

t_1, t_2 生存周期结束！——孩子节点、子表达式、运算对象
 t_3 生存周期开始！——父节点、运算结果

计算 $E_1 \rightarrow t_1$

计算 $E_2 \rightarrow t_2$

$t_3 = t_1 + t_2$
- 观察不同临时名字生存周期期间的关系
- 嵌套、前后，但不会交叉！
- → 栈！
- 产生式翻译：
 - 孩子节点临时变量弹出栈、归还临时名字池
 - 父节点从池中分配临时名字、压栈
- 不显式使用栈——计数器 c ，初始为 0
 - 临时名字作为运算对象使用： $c--$
 - 生成新的临时名字： $\$c, c++$

布尔表达式和控制流语句翻译

○ 布尔表达式的翻译就像是算术表达式的翻译

○ $a \text{ or } b \text{ and not } c \rightarrow$

$t_1 := \text{not } c$

$t_2 := b \text{ and } t_1$

$t_3 := a \text{ or } t_2$

○ $a < b \rightarrow \text{if } a < b \text{ then } 1 \text{ else } 0$

100: $\text{if } a < b \text{ goto } 103$

101: $t := 0$

102: $\text{goto } 104$

103: $t := 1$

104:

综合:

- 利用控制流语句传递来的“真”、“假”跳转地址生成转移语句；尚不能生成则继续传递给孩子节点

$E \rightarrow E_1 \text{ or } E_2$ $E_1.\text{true} = E.\text{true}; E_1.\text{false} = \text{newlabel};$
 $E_2.\text{true} = E.\text{true}; E_2.\text{false} = E.\text{false};$
 $E.\text{code} = E_1.\text{code} \parallel \text{gen}(E_1.\text{false} \text{ ':' }) \parallel E_2.\text{code}$

$E \rightarrow E_1 \text{ and } E_2$ $E_1.\text{true} = \text{newlabel}; E_1.\text{false} = E.\text{false};$
 $E_2.\text{true} = E.\text{true}; E_2.\text{false} = E.\text{false};$
 $E.\text{code} = E_1.\text{code} \parallel \text{gen}(E_1.\text{true} \text{ ':' }) \parallel E_2.\text{code}$

$E \rightarrow \text{not } E_1$ $E_1.\text{true} = E.\text{false}; E_1.\text{false} = E.\text{true}; E.\text{code} = E_1.\text{code};$

$E \rightarrow (E_1)$ $E_1.\text{true} = E.\text{true}; E_1.\text{false} = E.\text{false}; E.\text{code} = E_1.\text{code};$

$E \rightarrow \text{id}_1 \text{ relop id}_2$ $E.\text{code} = \text{gen}(\text{'if' id}_1.\text{place relop.op id}_2.\text{place}$
 $\text{'goto' } E.\text{true}) \parallel \text{gen}(\text{'goto' } E.\text{false})$

$E \rightarrow \text{true}$ $E.\text{code} = \text{gen}(\text{'goto' } E.\text{true});$

$E \rightarrow \text{false}$ $E.\text{code} = \text{gen}(\text{'goto' } E.\text{false});$

例子:

9. 将下面表达式转换为三地址码, 采用临时变量重用算法, 可将临时变量数目减少到 个。

$a + (a + a + a * (a + a + a))$

A. 2

B. 3

C. 4

D. 5

我们首先得清楚运算的顺序:

先是计算最右边的两个加法，然后再加，再乘，到这里都只需要使用一个临时变量。然后左边的和右边的相加需要使用一个临时变量，所以一共是两个

第九章 目标代码生成

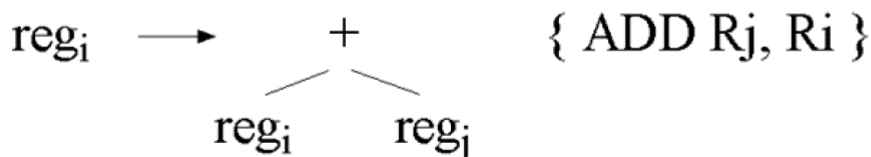
目标代码生成

- 目标代码生成算法
 - 基本块内局部最优
 - 计算结果尽量保存在寄存器内
- 寄存器分配优化
 - 着眼于循环，选定某些变量一直放于寄存器

代码生成算法

代码生成算法

- 改写规则——树→单个节点
 - `replace ← template { action }`
- 单个节点 树 语义动作
- 一组改写规则——树翻译模式
- 模板——机器指令，动作——翻译序列



- 输入树中子树与模板匹配→
将其替换为 reg_i ，输出 $\text{ADD } R_j, R_i$

寄存器分配

- 9.6节，基本块出口，寄存器→内存
 - 避免复制，将最常用的变量保持在寄存器中
 - 跨越基本块边界——全局
 - 循环
 - 使用固定一组寄存器保存每个内存循环中最活跃的变量
 - C运行程序员指定寄存器分配

例子：

9. 我们能在_____时完成数组越界判定。↵
- | | |
|-----------|------------|
| A. 类型检查 | B. 中间代码生成↵ |
| C. 目标代码生成 | D. 以上均不对↵ |

这题选

DAG构造

用DAG表示下面表达式，需_____个结点。
 $a + (a + a + a * (a + a)) * a$

- ☒ A 4
- ☐ B 5
- ☒ C 6
- ☐ D 7

代码优化

- 基本块（概念、如何划分）、流图（如何添加基本块的边）、循环（支配、回边）、下次引用（定义、引用等概念和使用方法（临时名字重用算法等））
- 基本优化方法
 - 消去公共子表达式
 - 复制传播
 - 无用代码删除
 - 循环优化
 - 代码外提
 - 强度削弱
 - 归纳变量删除

在进行代码优化的时候，重点考虑**循环**结构。

基本块的定义：连续的语句序列，执行过程中没有分支

1. 基本块构建——找leader

- 程序第一条语句（看题目有没有提到程序
- 无/有条件跳转**目的语句**
- 无/有条件跳转**下一条语句**（容易忘！

后面代码基本块不用显式构造 每个leader对应的基本块就是下一个leader或者是程序的尾部

2. 流图：前驱与后继

看一个基本块最后一个语句：

基本块 B_1 到 B_2 有一条有向边 $\leftarrow\rightarrow$ 代码执行序列中 B_2 紧跟在 B_1 之后：

1. B_1 的最后一条语句（无）条件转向到 B_2 的第一条语句
2. 程序中 B_2 紧跟在 B_1 之后，且 B_1 的最后一条语句不是无条件转移语句

➤ B_1 —— B_2 的前驱， B_2 —— B_1 的后继

- 有 有/无条件跳转到 B_2 —— B_2 （可能是false分支的无条件跳转） 1
- 无 有/无条件跳转，紧跟 B_3 —— B_3

后面代码基本块通过goto Lnext语句**指向空气**即可。

3. 循环：

- 这组节点是**强连通的**（任何两个节点之间都存在一条完全包含在循环内的路径）
- 这组节点具有**唯一的一个入口（entry）**——从循环外的节点到达循环内的节点，**唯一的途径是先到达入口节点**

4. 下次引用：从后往前扫描。

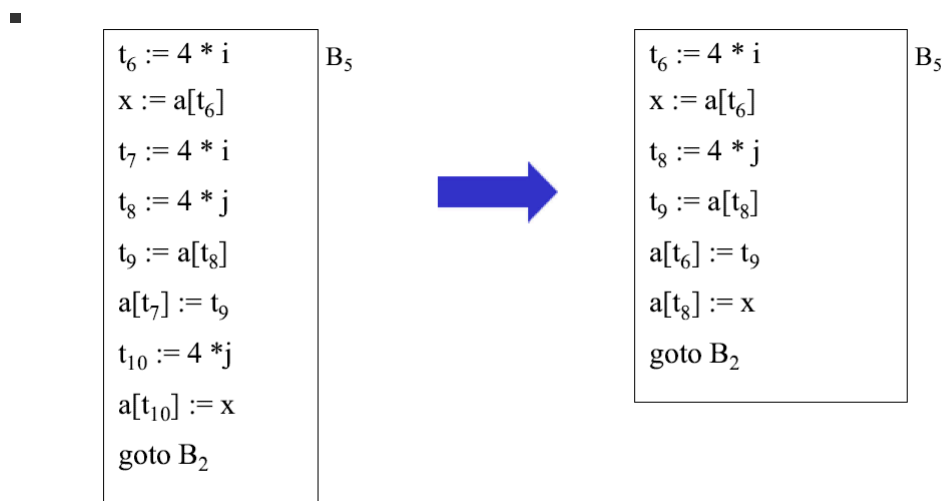
遇到指令i, $x = y \text{ op } z$:

- 在指令后加入引用信息（使用/引用(use)：语句赋值x, j以x作为运算对象，**且i到j无其他赋值语句**）
- x非活跃，无下次引用
- y活跃，有下次引用为i

后两步不可以交换。最后得到临时变量生存周期。数据流分析）临时变量名字的重用与存储分配。嵌套不可以，一上一下可以。

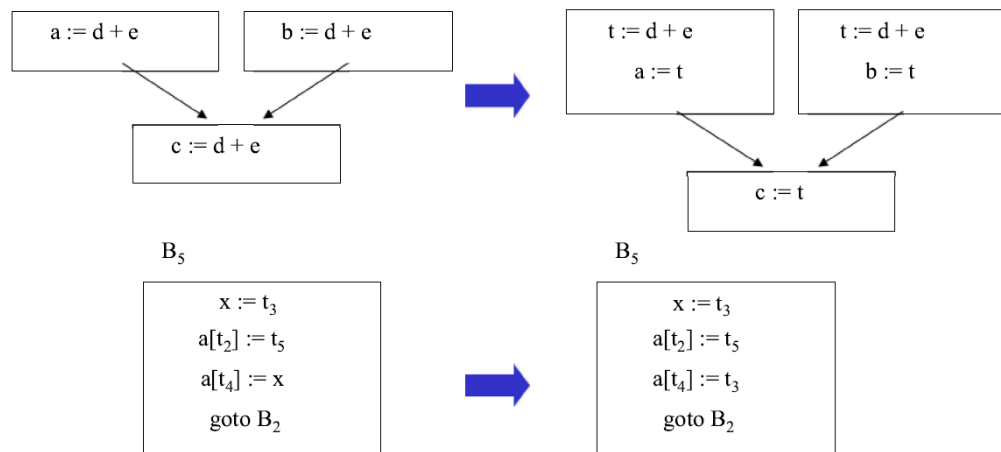
5. 基本优化方法(global与local):

- 消去公共子表达式（local）：要求两个重复的表达式之间没有对operand的定义（再赋值）。

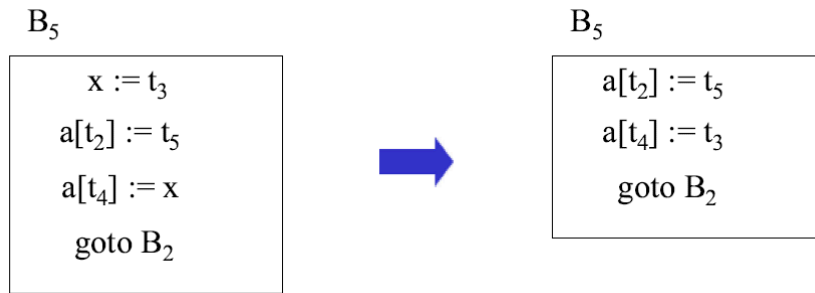


- 复制传播：

- $f := g$ ——在随后语句中，用g替换f



- 单路径传播：直接利用复制的源头（赋值语句的源头，即赋值语句的右式）
- 多路径传播：两个路径有相同右式的赋值表达式。不管从哪个路径来，用同一个临时变量代替。
- 无用代码删除：



- 利用复制传播，把变量直接替换为源头的常量（debug->false，也就做常量合并）。之后可以删除一些语句（源头的赋值语句，或者if(false)语句）
- **循环优化：**
 - 代码外提：提至循环外。

while (i <= limit - 2) ➔

t = limit - 2;

while (i < t)

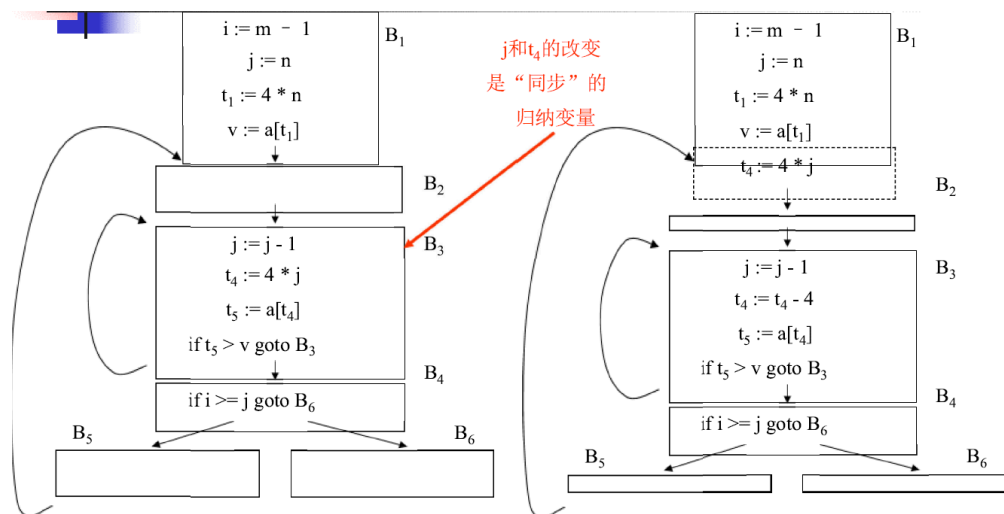
- 某些代码（表达式计算）与循环次数无关【或者说做的是相同的赋值操作】，即循环不变计算。

或者和循环条件有关的变量在循环内部没有对其的定义（赋值运算）——>循环不变计算。

eg. while(t <= limit - 2)

从循环内提到循环入口前。具体而言，提到一个前驱自定义基本块中。

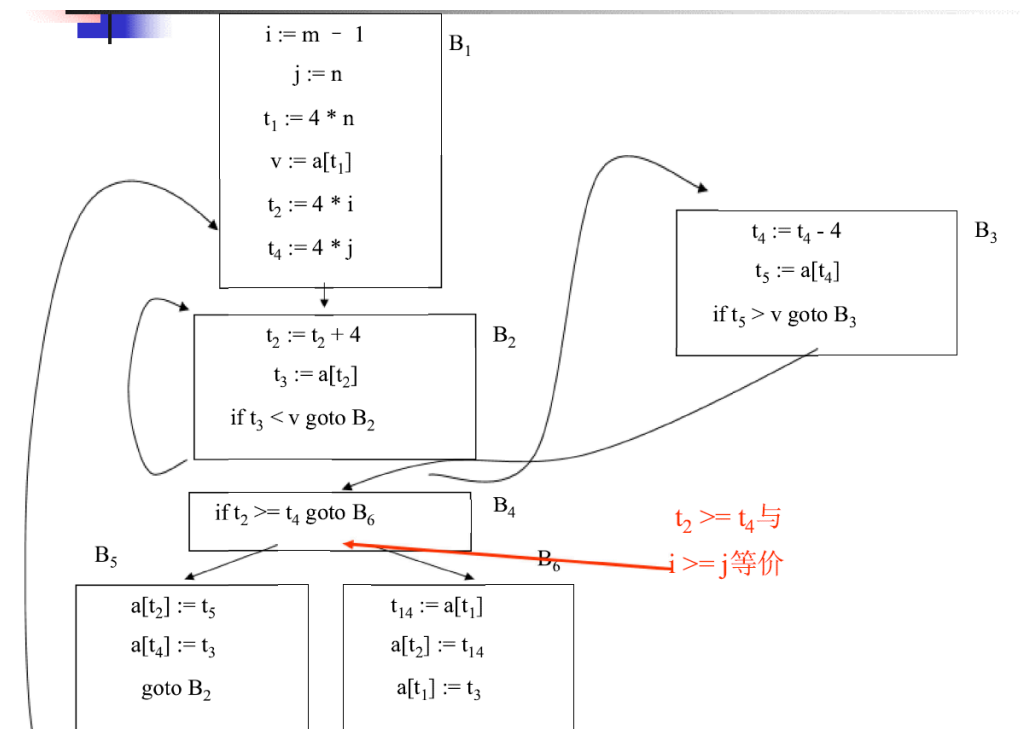
- **强度削弱：**与i, j, k多有关。乘方 > 乘除 > 加减。



- 同步变化的归纳变量，记住可能要多加使用某个归纳变量的赋值。

eg. j = j - 1; t4 = 4 * j ----> t4 = 4 * j; j = j - 1; t4 = t4 - 4;

- 归纳变量删除：归纳变量即i, j, k。



- 如果前面有归纳变量给临时变量的赋值，则用临时变量替换。（PPT）
- 如果之前没有，新加入临时变量的赋值，必须能减少一些。

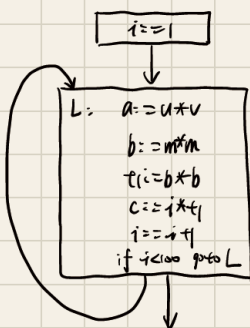
homework:

2. 将下面三地址码程序段划分基本块，画出流程图，然后进行代码外提、强度削弱、归纳变量删除等优化

```

i := 1
L:  a := u * v
    b := m * m
    t1 := b * b
    c := i * t1
    i := i + 1
    if i < 100 goto L
  
```

基本块划分+流程图



优化:

① 代码外提:

由于 a, b, t_1 的值不受循环影响，循环体前三句可以提到循环外边 ($a := u * v, b := m * m, t_1 := b * b$)

② 强度削弱:

$c := i * t_1$ ，当 i 递增 1 之后，相当于 $c = (i+1) * t_1 = c + t_1$ ，所以我们可以将 c 初始化为 0，用加法 $c = c + t_1$ 来代替，从乘法削弱到了加法，完成了强度削弱

③ 归纳变量删除:

循环一共执行了 99 次，所以可以将 i 删除，用 c 与 t_1 进行比较，控制循环，删除归纳变量 i 可以先声明一个 $t_2 := 99 * t_1$ ，然后判断 $c < t_2$ 即可。

得到结果:

优化结果:

